

Transformer-Based English to Hindi Translation

A From-Scratch Implementation Inspired by “Attention Is All You Need”

Kushagra Singhal

Roll No: 240150046

DA221-Introduction to Artificial Intelligence
IIT Guwahati

20 April 2026

Table of Contents

- 1 Introduction
- 2 Motivation
- 3 Methodology
- 4 Inference & Results
- 5 Conclusion
- 6 References

Introduction

Problem Setup

- **Objective:** Sequence-to-sequence neural machine translation from English to Hindi.
- **Mathematical Formulation:** Model the conditional probability of a target sentence y given a source sentence x :

$$P(y|x) = \prod_{t=1}^m P(y_t \mid y_{<t}, x)$$

- **Dataset:** 33,000 parallel EN-HI sentence pairs (30K train, 1.5K val, 1.5K test).

Approach

- Implementation of the **Transformer** architecture from scratch using PyTorch.
- Dispenses entirely with recurrence and convolutions, relying solely on **self-attention**.
- Incorporates SentencePiece Byte-Pair Encoding (BPE) to handle cross-lingual tokenization.

Limitations of Prior Architectures (RNNs/LSTMs)

- **Sequential Bottleneck:** Recurrent models process tokens sequentially (h_t depends on h_{t-1}), precluding parallelization within training examples.
- **Long-Range Dependencies:** Difficult to learn dependencies between distant positions in long sequences due to vanishing gradients.

Why the Transformer?

- **High Parallelizability:** Processes all sequence positions simultaneously, drastically reducing training time.
- **Constant Path Length:** Self-attention connects all positions with a constant number of operations ($O(1)$ sequential operations).

Methodology: Architecture & Configurations

Original vs. Implemented Configurations

Hyperparameter	Vaswani et al. (2017) Base	My Project
Embedding Dimension (d_{model})	512	128
Attention Heads (h)	8	4
Encoder/Decoder Layers (N)	6	2
Feed-Forward Dim (d_{ff})	2048	256
Max Sequence Length	5000	128
Vocabulary Size	~ 37000	20000 (BPE)

- **Hardware Adaptation:** The original model requires significant compute (8 GPUs for 12 hours to 3.5 days). This project scales parameters down to function efficiently on a 2 GB VRAM local GPU (NVIDIA RTX 2050) while retaining the core architectural integrity.

Methodology: The Encoder Process

Encoder Stack Architecture

- **Composition:** A stack of $N = 2$ identical layers.
- **Multi-Head Self-Attention:** Allows each position in the encoder to attend to all positions in the previous layer. No masking is required here.
- **Position-wise Feed-Forward Network (FFN):** Consists of two linear transformations with a ReLU activation in between:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- **Residual Connections & Layer Normalization:** Applied around each sub-layer (LayerNorm($x + \text{Sublayer}(x)$)) to stabilize deep training.

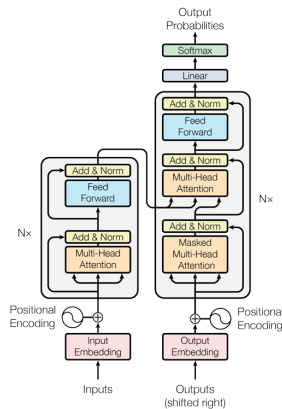


Figure: Transformer Architecture

Methodology: The Decoder Process

Auto-Regressive Generation

- Generates the target sequence $(y_1, \dots, y_m)$ one symbol at a time, consuming previously generated symbols as input.

Decoder Stack Architecture ($N = 2$ layers)

- **Masked Multi-Head Self-Attention:** Incorporates a target causal mask (an upper triangular matrix filled with $-\infty$). This prevents positions from attending to subsequent (future) tokens, preserving the auto-regressive property and preventing information leakage.
- **Encoder-Decoder Cross-Attention:** A crucial layer where the Queries (Q) come from the previous decoder layer, while the Keys (K) and Values (V) come from the final output of the encoder stack. This allows the decoder to align and attend over all positions in the input source sequence.
- **Position-wise FFN & Residuals:** Identical structure to the encoder, mapping representations to the final output vocabulary logits.

Methodology: Scaled Dot-Product Attention

Mathematical Formulation

- Maps queries (Q), keys (K), and values (V) to an output vector:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- Scaling Factor:** Dividing the dot product by $\sqrt{d_k}$ prevents the softmax function from entering regions with extremely small gradients.
- Facilitates highly optimized, parallelized matrix multiplications compared to additive attention.

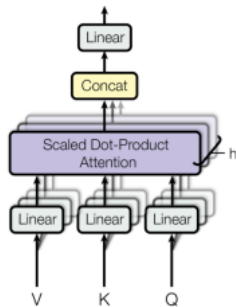


Figure: Scaled Dot-Product

Methodology: Multi-Head Attention

Parallel Attention Subspaces

- Linearly projects Q, K, V into h distinct sub-spaces (heads).
- Allows the model to jointly attend to different representation subspaces at different positions, which a single attention head cannot do due to averaging.
- **Configuration:** For our constrained implementation, we use $h = 4$ parallel heads, with $d_{model} = 128$ and $d_k = 32$.

Scaled Dot-Product Attention

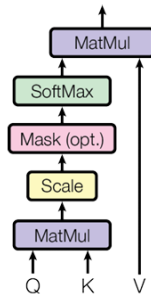


Figure: Multi-Head Attention

Methodology: Attention Visualizations

Learning Syntactic Dependencies

- Visualizing attention weights demonstrates how the model learns grammar and syntax without explicit programming.
- Different attention heads learn to perform different tasks, such as linking verbs to distant objects or resolving pronouns.
- This interpretability is a significant side benefit of the self-attention mechanism.

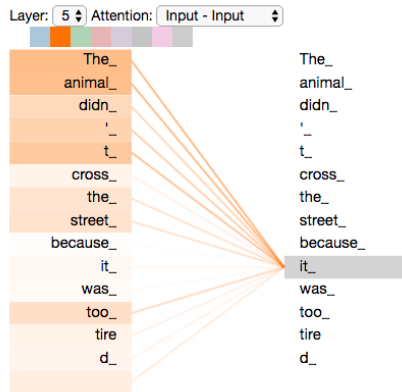


Figure: Attention Visualization

Methodology: Tokenization & Positional Encoding

SentencePiece BPE Tokenization

- **Strategy:** Byte-Pair Encoding (BPE) handles unknown words better than word-level mapping by breaking them down into recognizable subword pieces.
- **Vocabulary:** Maximum size of 20,000 tokens for the checkpoint. Includes special tokens (<pad>, <sos>, <eos>, <unk>).

Positional Encoding

- Injects sequence order using sinusoids since there is no recurrence. Summed directly with the learned token embeddings.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$



Figure: Positional Encoding

Methodology: Training Protocol & Optimization

Optimization Setup

- **Adam Optimizer:** $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$. Gradient clipping set to max norm 1.0.
- **Noam Scheduler:** Linearly increases learning rate for `warmup_steps` (400), then decreases proportionally to the inverse square root of the step count.

$$lr = d_{model}^{-0.5} \times \min(step^{-0.5}, step \times warmup_steps^{-1.5})$$

Regularization

- **Label Smoothing:** Cross-entropy loss with smoothing factor $\epsilon = 0.1$. Discourages overconfidence by uniformly distributing probability mass across non-target tokens.
- **Dropout:** Applied at a rate of 0.1 to residual connections and attention weights.

Inference & Decoding Strategies

Decoding Modes

- **Greedy Decoding:** Fast, auto-regressive generation that selects the top-1 token (highest probability) at each step. Suboptimal as greedy mistakes compound.
- **Beam Search ($k = 4$):** Explores multiple hypotheses simultaneously by tracking the k best partial sequences across steps.
- **Length Penalty ($\alpha = 0.6$):** Normalizes scores to prevent beam search from heavily favoring shorter sequences:
$$\text{score} = \log P(y) / (1 + \text{len}(y))^\alpha.$$

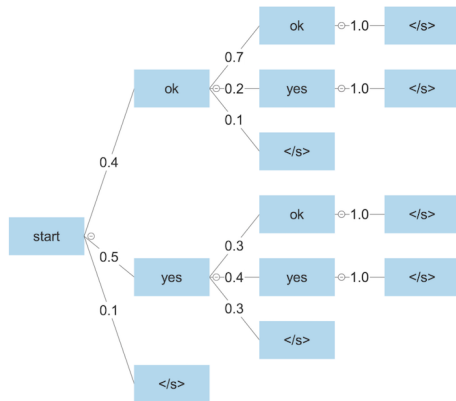


Figure: Beam Search Decoding

Results: Benchmarks and Metrics

Experimental Setup

- **Hardware:** NVIDIA RTX 2050 (2 GB VRAM).
- **Training run:** 20 Epochs on 30,000 synthetic pairs. Batch size: 32.
- **Evaluation:** 256-sample test benchmark.

Empirical Results

Decoding Method	BLEU-4 Score	Runtime (s)	Samples/s
Greedy Search	7.22	47.44	5.40
Beam Search ($k = 4, \alpha = 0.6$)	7.22	103.43	2.48

- The BLEU score reflects the constraints of $d_{model} = 128$ and a 30K dataset, which lacks the representational capacity for deep syntax, but functionality is validated.
- Achieved a highly efficient throughput of 5.40 samples/s via Greedy Search on constrained hardware.

Conclusion, Limitations & Future Scope

Conclusion

- Successfully engineered a complete, from-scratch Transformer pipeline spanning BPE tokenization, sequence-to-sequence modeling, and multi-strategy inference.
- Validated the mathematical mechanics of scaled dot-product attention and Noam scheduling within a constrained hardware environment.

Limitations

- Heavily bottlenecked by the 2 GB VRAM limit, necessitating reduced parameter counts.
- Synthetic dataset of 30K pairs limits vocabulary generalization and syntactic depth.

Future Scope

- Integrate larger parallel corpora, such as the IIT Bombay English-Hindi Corpus.
- Implement gradient checkpointing and mixed-precision (FP16) training to scale up to the standard $d_{model} = 512$ architecture.

References



Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017).

Attention is all you need.

Advances in Neural Information Processing Systems, 30.



Sennrich, R., Haddow, B., & Birch, A. (2015).

Neural machine translation of rare words with subword units.

arXiv preprint arXiv:1508.07909.



Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016).

Layer normalization.

arXiv preprint arXiv:1607.06450.